

作业1：cc (codex) 结合 05-multimodal-rag-chatbot 的需求，vibe coding

我要重新实现一个多模态 RAG 图文问答项目。请不要直接写成几个全局脚本，而是按可测试、可替换外部服务的架构实现。

项目目标：
用户上传 PDF/docx/txt 文档，系统保存文件和元信息，投递离线解析任务。worker 解析文档为 markdown 和图片，切分 markdown chunk，生成文本/图片向量并写入向量库。聊天时，用户输入问题，系统检索相关 chunk，把检索内容和来源拼入 prompt，调用 LLM 生成答案，并支持 markdown 图片渲染。

技术约束：

- 1. Web UI 使用 Streamlit。
- 2. 元数据存储使用 SQLite + SQLAlchemy。
- 3. 队列层需要设计接口，搭起 Kafka adapter 框架。
- 4. 向量库层需要设计接口，搭起 Milvus adapter 框架。
- 5. Embedding 层需要设计接口，暂定使用clip embedding。
- 6. LLM 层需要设计接口，设计 OpenAI/DashScope adapter。
- 7. Parser 层需要设计接口，默认 fake parser，另外预留 mineru adapter。
- 8. 配置必须从环境变量或配置文件读取，禁止硬编码 API key、token、模型路径、服务地址。
- 9. Kafka、Milvus、mineru、LLM 先搭起框架。...

Show more

9:51

Worked for 1m 32s

我会先按“分层架构 + 可替换 adapter + 核心逻辑可单测”的方式规划。当前工作区除了 Serena 元数据外没有业务代码，因此下面是从零实现的建议结构；本轮只输出计划和文件结构，不写代码。

总体方案

层级	目标	关键原则
UI 层	Streamlit 上传文档、查看任务状态、发起聊天、渲染 markdown 图片	只调用 service，不直接碰数据库、Kafka、Milvus、LLM SDK
Application Service 层	编排上传、离线解析、索引、检索、问答流程	只依赖接口，不依赖具体外部服务实现
Domain/Core 层	定义业务实体、任务状态、chunk、检索结果、prompt 构造规则	无外部 SDK，方便 pytest 覆盖
Ports 接口层	定义 Queue、VectorStore、Embedding、LLM、Parser、Storage 等抽象接口	外部服务可替换，测试可用 fake/in-memory 实现
Adapters 实现层	Kafka、Milvus、CLIP、OpenAI、DashScope、mineru、SQLite、本地文件存储等实现	adapter 内部处理 SDK 细节，核心业务不感知
Worker 层	消费解析任务，执行 parser -> chunk -> embedding -> vector upsert -> DB 状态更新	可以本地 fake queue 跑通，也可替换 Kafka
Infrastructure 层	配置、日志、数据库 session、依赖装配	API key、模型路径、服务地址全部从环境变量或配置文件读取

建议文件结构

text

作业1/

```
|— AGENTS.md
|— README.md
|— pyproject.toml
|— .env.example
|— config.example.yaml
|— data/
|   |— uploads/
|   |— parsed/
|   └─ images/
|— src/
|   └─ multimodal_rag/
|       |— __init__.py
|       |— config.py
|       |— logging_config.py
|       |— bootstrap.py
|       |— domain/
|       |   |— __init__.py
|       |   |— entities.py
|       |   |— enums.py
|       |   └─ errors.py
|       |— ports/
|       |   |— __init__.py
|       |   |— queue.py
|       |   |— vector_store.py
|       |   |— embeddings.py
|       |   |— llm.py
|       |   |— parser.py
|       |   |— storage.py
|       |   └─ repositories.py
|       |— db/
|       |   |— __init__.py
|       |   |— base.py
|       |   |— session.py
|       |   |— models.py
|       |   └─ repositories.py
|       |— adapters/
|       |   |— __init__.py
|       |   |— queue/
|       |   |   |— in_memory.py
|       |   |   └─ kafka.py
|       |   |— vector_store/
|       |   |   |— in_memory.py
|       |   |   └─ milvus.py
|       |   |— embeddings/
|       |   |   |— fake.py
|       |   |   └─ clip.py
|       |   └─ llm/
```



作业2：阅读mineru 论文和 mineru 的使用；尝试进行文档解析，对比下mineru和 pdfplumber的效果差异

- 论文的关键点可以概括为：MinerU2.5 不是直接把整页高分辨率图片扔给大模型，而是先低成本看全局版面，再对重要局部进行高分辨率识别。这正好解决论文、报告、扫描件这类文档的典型问

题：页面大、元素多、公式和表格细节密集，直接整页识别成本高且容易丢细节。

- MinerU 更像“把文档还原成可读 Markdown”；pdfplumber 更像“把 PDF 对象取出来给程序处理”。
- 论文类、扫描类、混排类文档 MinerU 通常明显更稳。
- MinerU 更适合直接生成可读文档；pdfplumber 更适合程序化抽字段。
- 科研论文里 MinerU 对公式保真明显更合适。
- 如果你要做精细规则抽取，pdfplumber 更灵活。